

2DX3 Final Deliverable Report

McMaster University

Mawa Hassan – hassm7 - 400522175

Date: April 7th, 2026

Statement of Originality: As a future member of the engineering profession, the author is responsible for performing the required work in an honest manner, without plagiarism and cheating. Any ideas or techniques from the work of others are fully acknowledged in alignment with standard referencing practices. Submitting this work with my name and student number is a statement and understanding that this work adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario.
Submitted by [Mawa Hassan, hassm7, 400522175]

Device Overview

Features

- MSP432E401Y Microcontroller with 26MHz configured bus speed.
- Onboard memory consists of integrated SRAM and Flash (TM4C1294)
- Operating voltage of 3.3 V to 5 V depending on component.
- UART communication of 115200 baud rate.
- I2C serial communication between Time-of-Flight (ToF) sensor and microcontroller.
- 3D output visual of scanned data using MATLAB R2025b.
- 360° scanning ability with 11.25° measurement segments.
- On-board pushbuttons (PJ0, PJ1) for user control
- Approximate cost: \$260 CAD.

General Description

This design consists of an embedded microcontroller that utilizes a Time-of-Flight (ToF) sensor to deliver 3D spatial mapping using collected distance measurements of an indoor space. The Time-of-Flight sensor works by emitting an infrared light signal, and then measuring the time taken for the signal to be reflected and for it to return to the sensor. This process allows for distance measurements to be taken on a millimeter scale. The sensor also contains an analog-to-digital converter within its framework as to simplify integration with other embedded devices, which, in this case, is the MSP432E401Y microcontroller.

In this design, the sensor is mounted onto a stepper motor, which allows the sensor to be rotated through angular segments of 11.25 degrees of a full 360-degree scan. The stepper motor itself is configured through GPIO output pins and a driver board. A stepper motor is used as to ensure accurate intervals within angle measurements. Once the stepper motor rotates the sensor to the correct angle, the ToF sensor then sends distance data to the microcontroller via Inter-Integrated Circuit (I2C) communication protocols and derives the corresponding angle and distance measurements. These values are then stored in memory. After a successful scan, the collected measurements are then transmitted to the connected personal computer (PC) using Universal Asynchronous Receiver-Transmitter (UART) serial communication at a 115200 baud rate. This structure allows for multiple scans to be transmitted before they are then further processed.

Once the data is sent to the PC, MATLAB is used to receive this data and then output it in a visual format. The data that is input is in polar coordinate form (angle and distance) and is then converted into Cartesian coordinate form (x,y,z) so that it may be plotted onto a 3D graph. The measurements the ToF sensor takes are plotted onto the y and z axis using simple trigonometry, while the x-axis uses the assumption that the entire mechanism is being moved with a consistent displacement across the x-axis. This information is then plotted using MATLAB's 3D plotting functions, as a way to visualize the environmental scan and ensure the accuracy of said scan.

Block Diagram

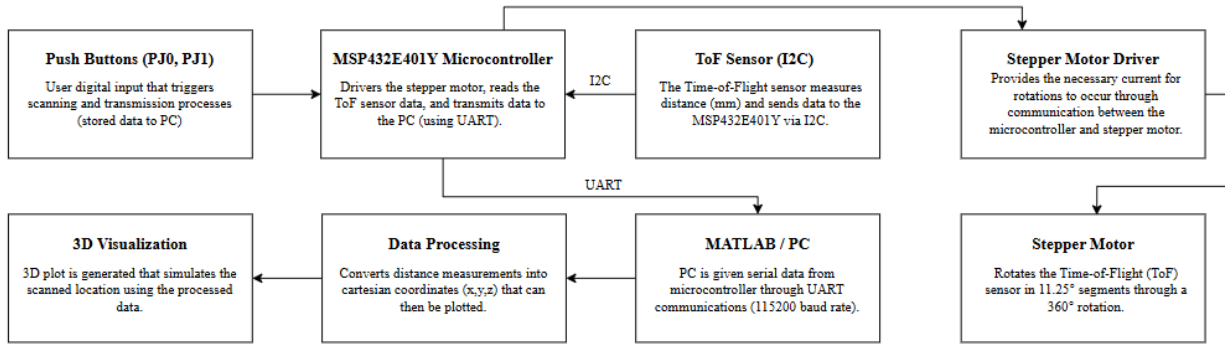


Figure 1: Block Diagram.

Device Characteristics Table

Table 1: Device Characteristics Table.

Microcontroller		ToF Sensor		Stepper Motor System	
System Clock (PLL Output)	60 MHz	Type	VL53L1X	Step Size	11.25 °
Bus Speed (Configured)	26 MHz	Range	~4 m	Control Pins	PH0 – PH3
Push Button 1 & 2	PJ0, PJ1	Resolution	1 mm	Driver	ULN2003
UART Baud Rate	115200 bps	Communication	I2C (100kHz)	Stepper Motor Driver Board Pin Mapping	
PC Software	MATLAB	Vin	3.3V		
Serial Port	COM4	SDA	PB3	IN1-IN4	PH0-PH3
Status LEDs	PN0, PN1, PF4	SCL	PB2	V+	5 V
				V-	GND

Detailed Description

Distance Measurement

The design implemented uses a Time-of-Flight (ToF) sensor of VL53L1X type to measure the distance between the sensor the surrounding environment. The sensor works by emitting an infrared pulse laser and measuring the amount of time it takes for the reflected signal to return back to the sensor. The sensor then calculates the distance using this timing measurement and the speed of light, yielding the value of the distance travelled in millimeters. The equation for this is below:

$$\text{Measured Distance} = \frac{\text{Photon Travel Time}}{2} \times \text{Speed of Light}$$

The photon travel time is the amount of time it takes for the light signal to be emitted and reflected back to the sensor, and the speed of light is a constant. The reason the travel time is divided by 2 is to account for the fact that the time taken for the ToF sensor is for a round-trip, and not a one-way endeavour. After the sensor performs the calculation, it then outputs that value in millimeters to the microcontroller via I2C serial communication. This greatly simplifies the process as the ToF sensor handles a large aspect of the analog-to-digital conversion process internally. This communication between the sensor and the microcontroller via I2C occurs at roughly 100kHz, where the microcontroller repeatedly polls the sensor as a means to check if there is any new data. After some data has been collected by the sensor, the microcontroller reads that value from the sensor and then clears the interrupt to prepare for another measurement.

As a way to collect data in a 360-degree rotation, the Time-of-Flight sensor is mounted onto a stepper motor, where it rotates along with the direction of the motor. The motor is configured to move in 11.25-degree increments, allowing for the stepper motor to take a measurement at each section. The motor rotates through a full 360 degrees, which implies that 32 measurements are taken per revolution. Multiple scans are stored in memory, with the number of maximum scans being configurable by the user. However, the default number of scans by the system is 8.

The data the Time-of-Flight sensor collects is in polar coordinate form, as it contains a distance measurement as well as an angular aspect. When this data is transmitted to the PC, it is converted into Cartesian coordinate form where it can then be plotted using MATLAB's 3D plotting functions. As mentioned, this conversion from polar to cartesian form occurs on the PC's side, but this can only be done by first receiving the data through UART serial communication. Each distance and angle measurement is used to deliver the position on the y-z axis, whereas the x-axis measurements are computed through the assumption that there is a constant displacement between x-coordinate scans. The y-z plane positions are delivered through the use of simple trigonometric functions. This is through the assumption that the angle of rotation on the sensor corresponds with the vertical plane. The x-coordinate is assigned based on the scan index, representing the displacement between successive scans. The exact equations used to convert from polar to Cartesian coordinates are indicated below:

$$\begin{aligned} r &= \text{measured distance (mm)} \\ \theta &= \text{angle (degrees)} \\ d &= \text{displacement between scans (mm)} \\ n &= \text{scan index} \end{aligned}$$

$$x = n \times d$$

$$z = r \times \sin(\theta)$$

To further explain the conversion process, an example measurement can be used where the distance the ToF sensor has measured is 1000mm (100cm), with an angle of 90 degrees, during the second scan ($n = 1$), and a fixed x-displacement of 100mm (10cm).

$$x = 1 \cdot 100 = 100 \text{ mm}$$

$$y = 1000 \cdot \cos(90^\circ) \approx 0 \text{ mm}$$

$$z = 1000 \cdot \sin(90^\circ) \approx 1000 \text{ mm}$$

This calculation provides a Cartesian coordinate measurement which can then be plotted as a point in the full 3D plot.

As showcased previously, the x-displacement is implemented through implied movement in the overall design rather than through the actual motion of the sensor. The x-displacement is assumed to be moving in a linear fashion between each scan, fully allowing the y-z scans taken by the Time-of-Flight sensor to be implemented into a 3D plot. In the MATLAB program, a value of 100mm is used between scans, indicating that the user must move 10cm between each measurement. This is achieved by incrementing the x-coordinate for each scan index, effectively stacking multiple 2D segments to create a 3D plot. By using this approach, the hardware aspects of the implementation are simplified while still creating a relatively reliable 3D plot of the environment.

Visualization

The way in which the 3D environment is visualized is through the use of MATLAB. In MATLAB, serial communication is established through the use of the serialport function. This function reads the data coming from the microcontroller, which is through UART at a baud rate of 115200 bps. Each scan consists of the angle and distance measurements which are stored in arrays, and where invalid measurements or measurements of zero are discarded. As previously mentioned, using sin and cos functions, the data is then converted from polar to cartesian coordinates. The actual 3D plot is generated using MATLAB's plot3() function. In this function, each scan is plotted using a connected number of points. Afterwards, lines are drawn between each corresponding scan to more accurately replicate the 3D structure of the scanned environment. Each axis is on the millimeter scale, as to accurately recreate a plot of the scanned area.

Application Note, Instructions, and Expected Output

Application Note

The design is implemented as a way to map the 3D environment of a given location using embedded spatial mapping. The Time-of-Flight (ToF) sensor is mounted onto a stepper motor as a means to collect the distance measurement at different 11.25-degree angular segments of a 360 scan. A more practical environment for this design is through the scanning of indoor locations; including, but not limited to, hallways or classrooms. In this application, the design is used to map a hallway location through the same mechanism explained above. Every time the sensor captures a scan, it is recording the measurements in the y-z plane, and then integrating the built-in x-displacement measurements that can be specified by the user. Through the combination of

these measurements, a full 3D reconstruction of the environment can be achieved. This project can be useful to a variety of different applications. For example, in robotics it could be beneficial in designs where obstacle avoidance or path planning is essential. Furthermore, since the project is on the lower end when it comes to cost, it can be used in place of more typical LiDAR systems. This simplified implementation still demonstrates the main aspects of spatial data acquisition and visualization.

Instructions

The following steps outline the process on how to set up and operate the 3D mapping sensor system. These instructions use Keil uVision MDK-Arm Version 5.43a and MATLAB R2025b.

1. Begin by setting up the hardware.
 - a. Connect the microcontroller to the PC using a USB cable.
 - b. Connect the ToF sensor via I2C and the corresponding pins from the ToF sensor to the microcontroller (SDA→PB3, SCL→PB2, GND→GND, VIN→3.3V)
 - c. Connect the stepper motor driver and the stepper motor. Then connect the driver to the microcontroller through the corresponding pins (IN1-IN4→PH0-PH3, V+→5V, V-→GND)
2. Ensure the correct COM setup.
 - a. Open device manager on the PC.
 - b. Expand the Ports (COM & LPT) section.
 - c. Find the Port with “XDS110 Class Application/User UART (COM#)” and note this number.
 - d. Open the MATLAB script and on line 6, change the COM# to the number noted in step 2c.
 - e. Ensure that the baud rate is set to 115200 bps.
 - f. Check the ‘expectedScans’ value on line 10 to ensure the number of scans you are intending is written there. Change the value if it is inaccurate.
3. Load Keil uVision program.
 - a. Open the program in Keil uVision, and ensure the MAX_SCANS value corresponds with the value written in the MATLAB ‘expectedScans’ value, or that it is greater than the value in MATLAB.
 - b. press Translate → Build → Load.
 - c. Press the reset button on the microcontroller board to reset the system.
4. Collect data.
 - a. Run the MATLAB program and ensure the prompt that indicates the program is ready for incoming data appears.
 - b. Press the scan button (PJ1) to perform a full rotational scan of the environment.
 - c. Ensure during this scanning process that the stepper motor rotates in 11.25-degree increments, and that the measurement LED (PN0) flashes for each reading that is completed.
 - d. Repeat this process (Press PJ1) for the remaining scans.
5. Process the data.
 - a. After all the scans have been completed, press the process data button (PJ0). This will send all the stored scans to MATLAB (using UART).
 - b. Ensure the UART status LED (PN1) flashes during this transmission to ensure the data is transmitted.

6. 3D Visual.

- a. After you press PJ0, all the data will automatically be sent to MATLAB and converted into a 3D plot using the converted Cartesian coordinates. This plot represents the 3D scanned environment.

Expected Output

After all the aspects of the system are appropriately configured, the expected output should be a 3D plot of the scanned area. The plot will be created in MATLAB using the data the Time-of-Flight (ToF) sensor collected. Each scan by the ToF sensor produces a 360-degree set of y-z coordinates, which indicate the distance of the Time-of-Flight sensor from nearby obstacles (objects, walls, etc.). Once more than one scan is collected, the design will implement the previously defined x-displacement between each y-z scan, creating a full 3D representation of the scanned environment. In this completed plot, the x-axis represents the displacement between each scan, and the y and z axes represent the measured distances by the ToF sensor. The overall shape of the environment should replicate the environment that was scanned. For example, if a square box was scanned, the outputted 3D plot should be a structure that represents a cube-like structure. In a more complicated scan, such as a hallway, any obstacles should appear as distortions or protrusions. Furthermore, they may also be some inaccuracies due to slight noise. This occurs due to limitations in the hardware implementation as the sensor may experience some resolution inaccuracies, and the motor's angle may not be completely precise (if the motor is not set up exactly on the positive y-axis).

An example of the expected output can be seen below. The design was used to map a hallway, using a displacement of roughly 200mm (20cm) between scans. As intended, the scan captures the main features of the hallway; areas in which the hallway becomes narrower are showcased, and the overall structure of the hallway is clear.

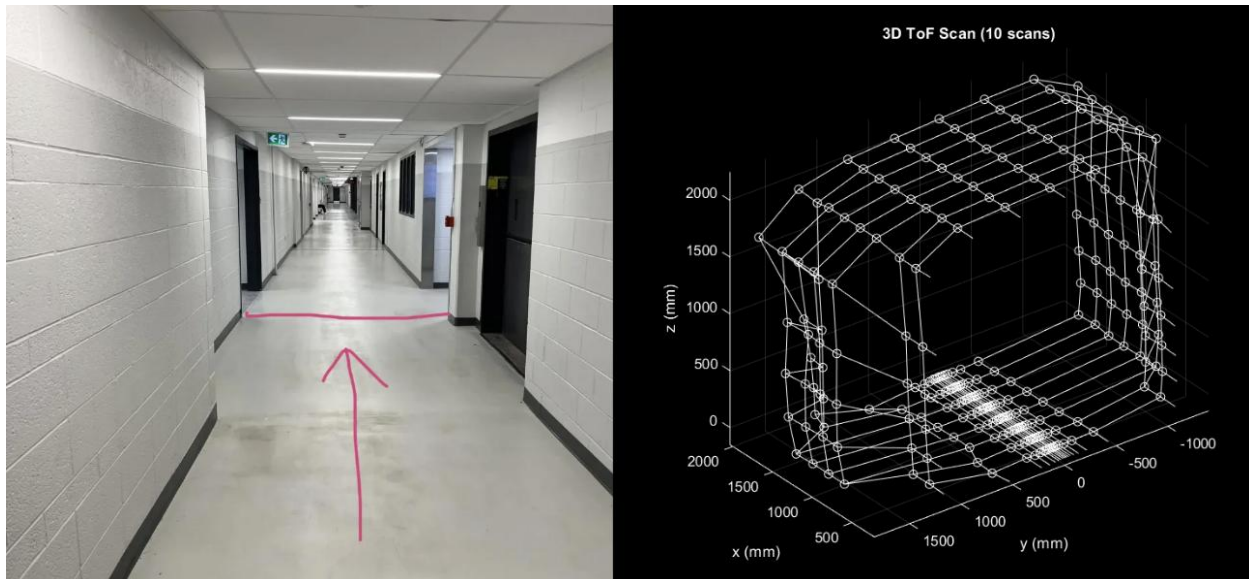


Figure 2: 3D plot of designated hallway.

Limitations

Although the system reliably outputs a 3D scan of the scanned environment, there are still some limitations in the implementation. For one, there are some limitations when it comes to the floating-point capability and implementation of the trigonometric functions. This is because floating-point calculations are not completely accurate due to the lack of precision. This happens when the data is being converted into Cartesian form in the MATLAB aspect using the trigonometric functions. Oftentimes, these functions output values that are extremely long decimal values, but are rounded which can occasionally produce inaccuracies or errors. These inaccuracies can then lead to the final coordinate values being a bit off, leading to the 3D plot being distorted in comparison to the actual scanned environment. Secondly, the maximum serial communicate rate available is a 128000 baud rate. In this implementation of the design, a baud rate of 115200 bps was used for communication between the MATLAB software and the microcontroller. This value was ensured in a couple of ways. For one, by confirming that the serial port on the PC supported standard rates up to 128000 bps, and second, by successfully receiving complete scan data in MATLAB. The MATLAB receiver code was configured to use 115200 bps, matching the UART transmission settings of the microcontroller. Another limitation was the serial communication method used (I2C) and the speed between the microcontroller and the ToF module. In this implementation, the I2C speed was roughly 100kHz, which was set through the I2C timing register. In this register, the values chosen were to ensure that the I2C clock has a speed of roughly 100kHz for the designated bus speed of 26 MHz. Lastly, there was some limitations in the speed of the system. This occurs during the rotation of the stepper motor and delay between each motor phase change. Additionally, there is a limitation when the sensor takes a measurement and the actual ready time of the system. This occurs due to the fact that the stepper motor cannot be given an excessively large speed as to avoid missteps or unstable motion, as a large enough speed will cause the stepper motor to not function. Furthermore, the current implementation requires the Time-of-Flight sensor to indicate to the program that new data is ready before it can be taken which produces more delay in the program. These limits were tested by observing the operation of the system while the mentioned delays were in place. By doing this, stable rotation and successful scans were collected and analyzed. The motor timing was also tied to the configured system clock, and the communication speed was checked through successful UART transmission to MATLAB. Altogether, these tests showcase that the speed of the entire system is limited by a number of parts; such as the motor timing, sensor readiness, and serial data transfer.

Circuit Schematic

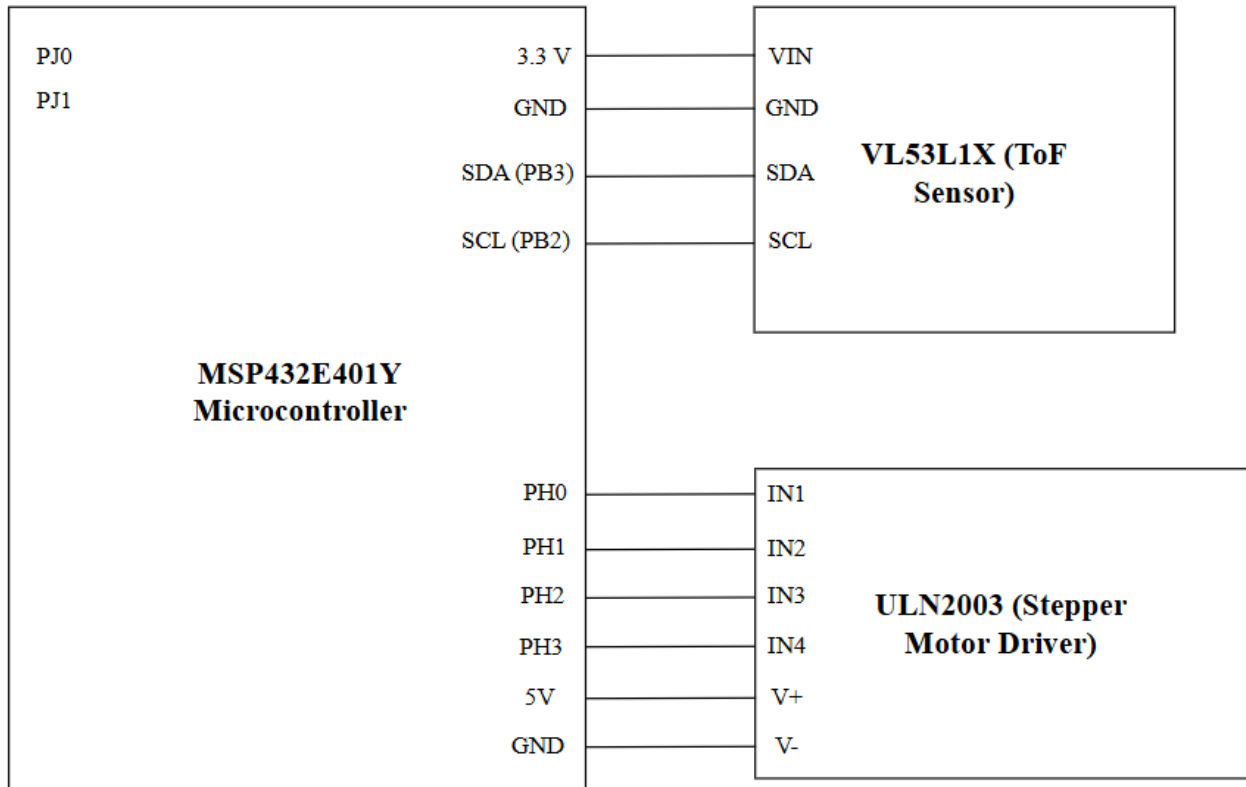


Figure 3: Circuit Schematic.

Programming Logic Flowchart(s)

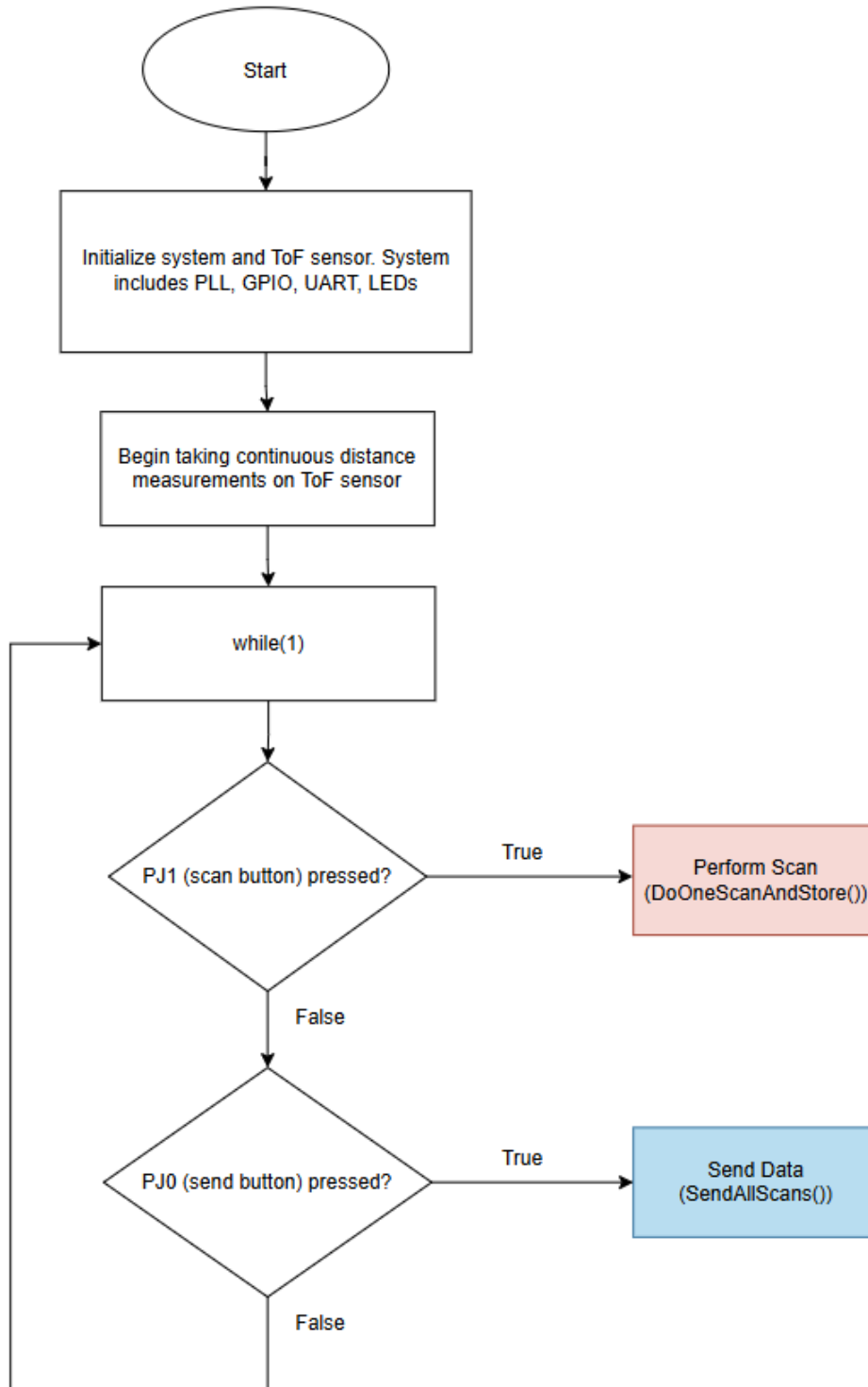


Figure 4: Programming Logic Flowchart Keil Program.

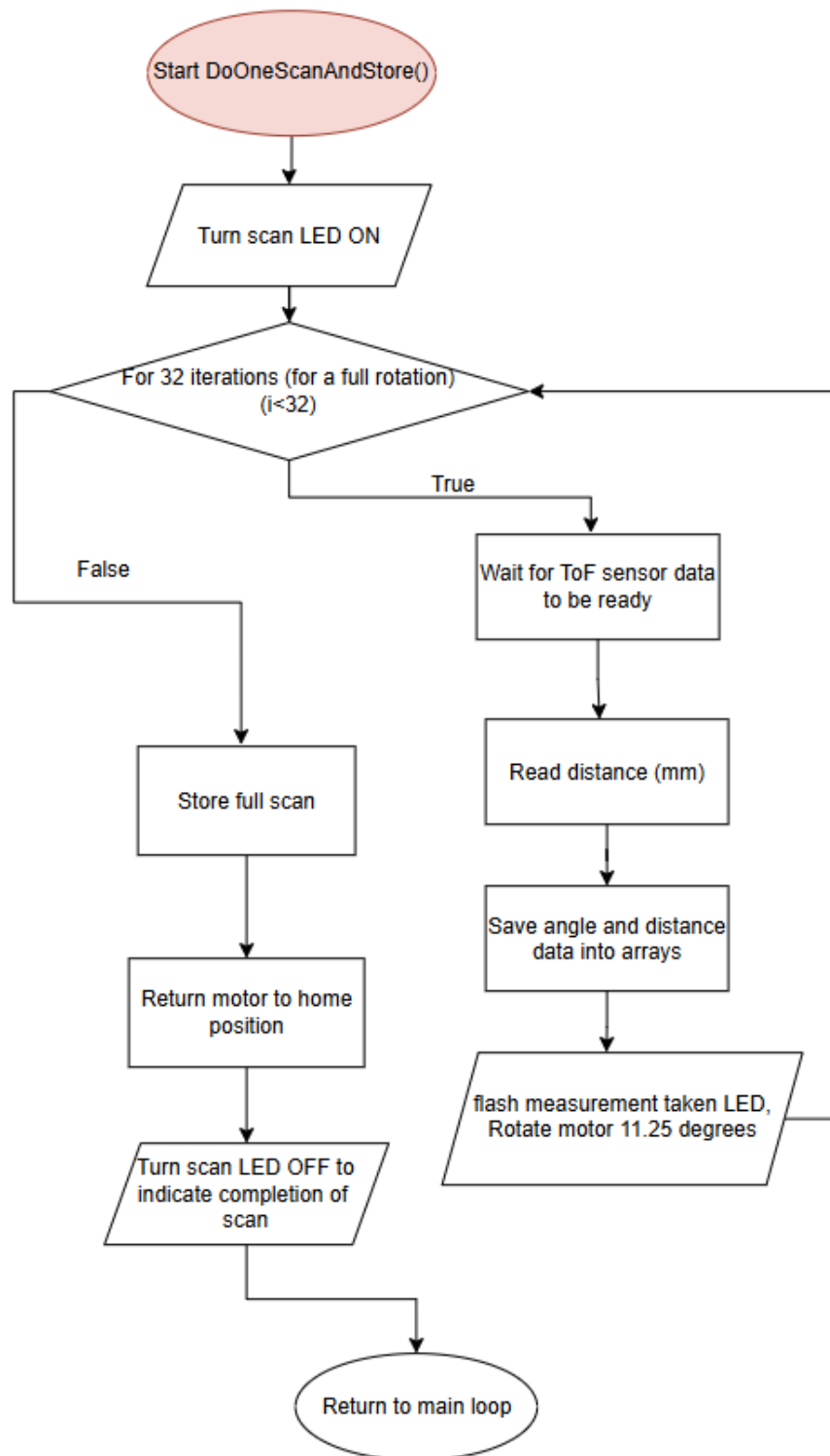


Figure 5: Programming Logic Flowchart Keil Program 2.

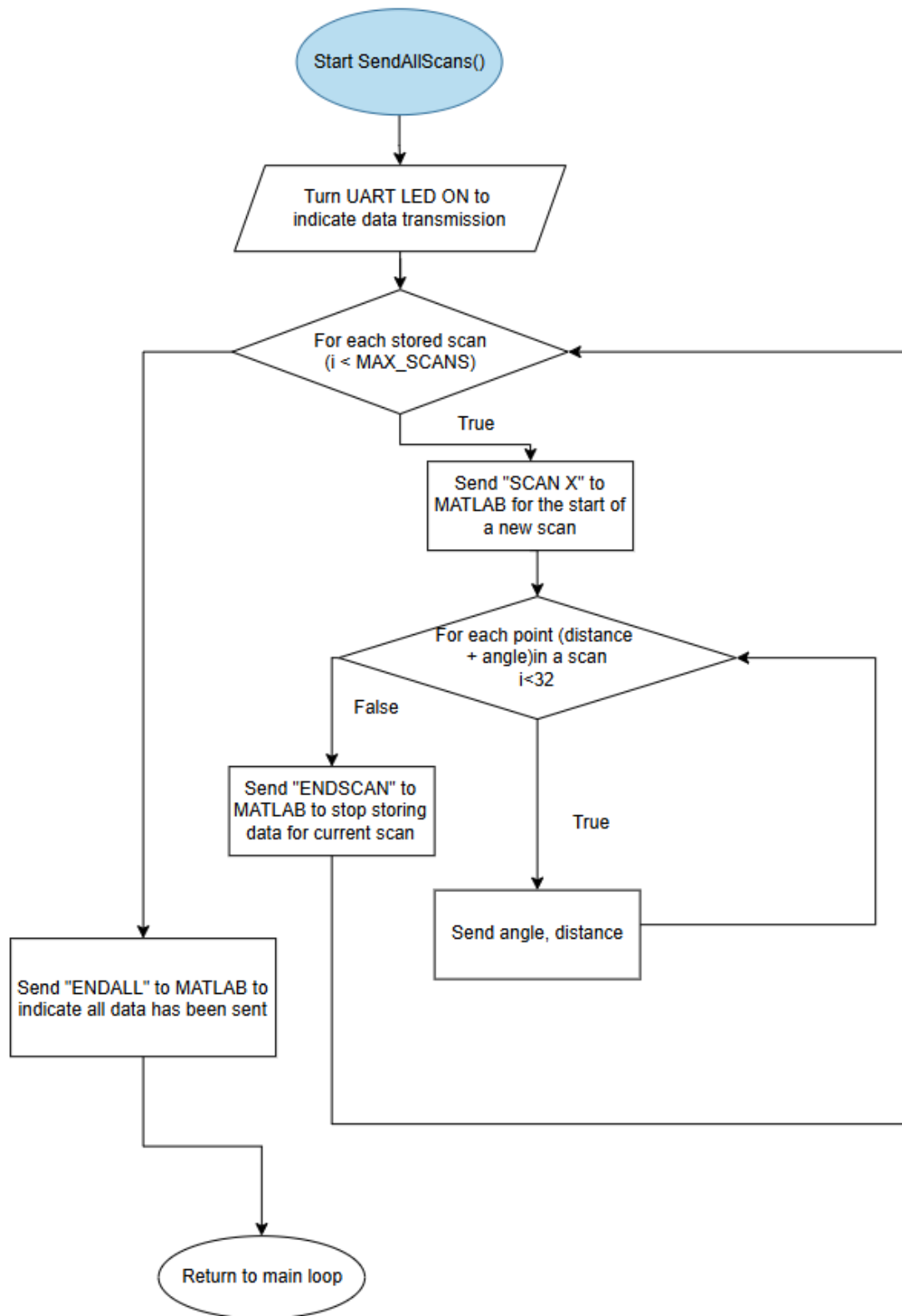


Figure 6: Programming Logic Flowchart Keil Program 3.

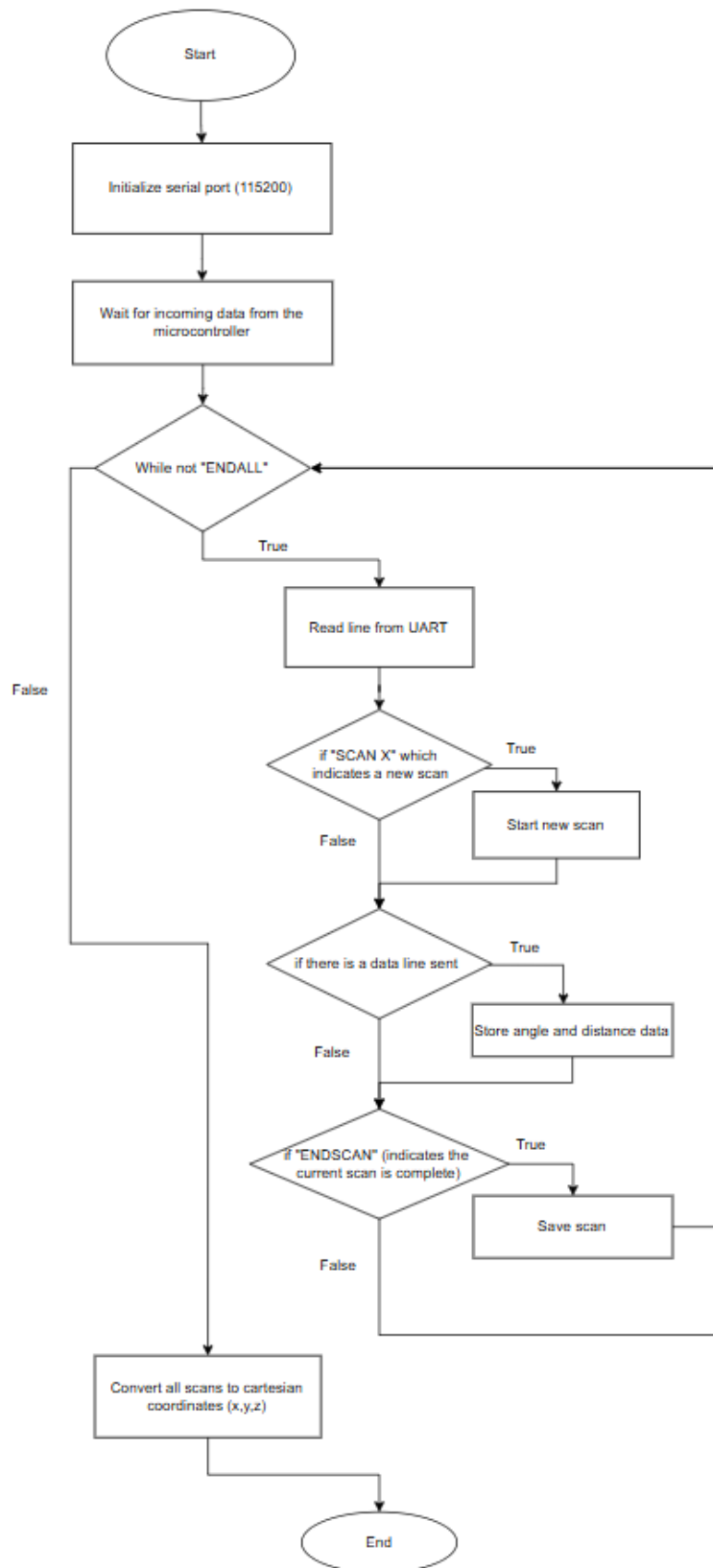


Figure 7: Programming Logic Flowchart MATLAB Program.